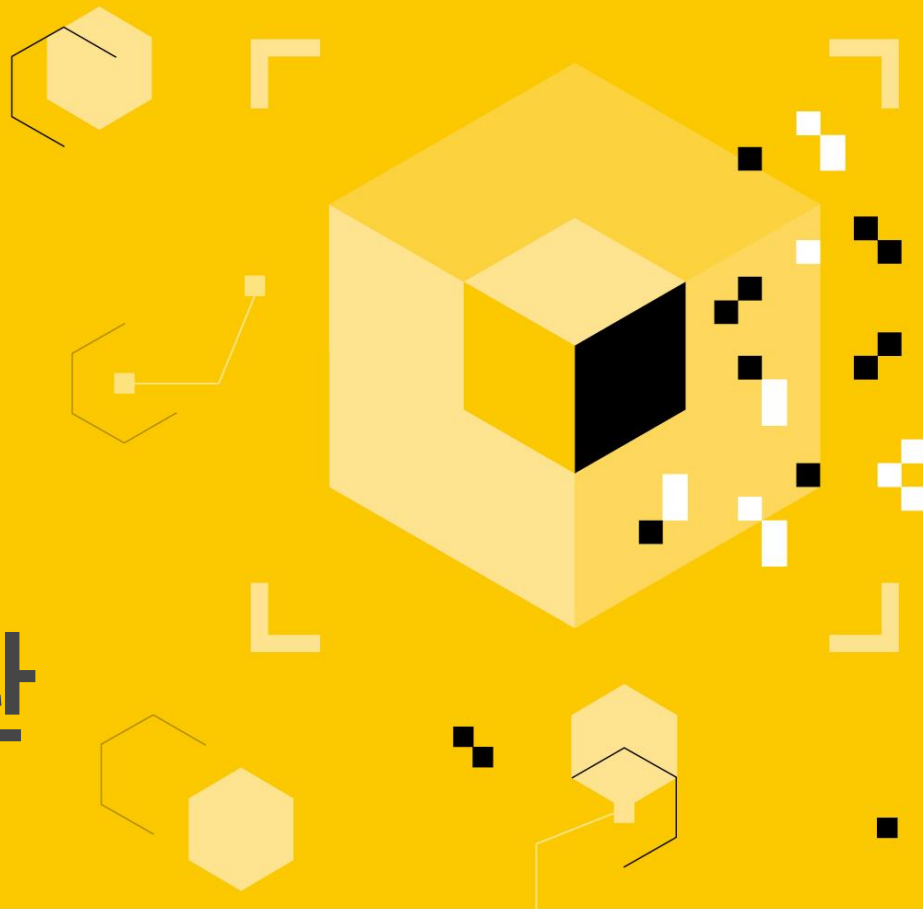


CHAPTER 08

영상의 기하학적 변환



OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

8장 영상의 기하학적 변환

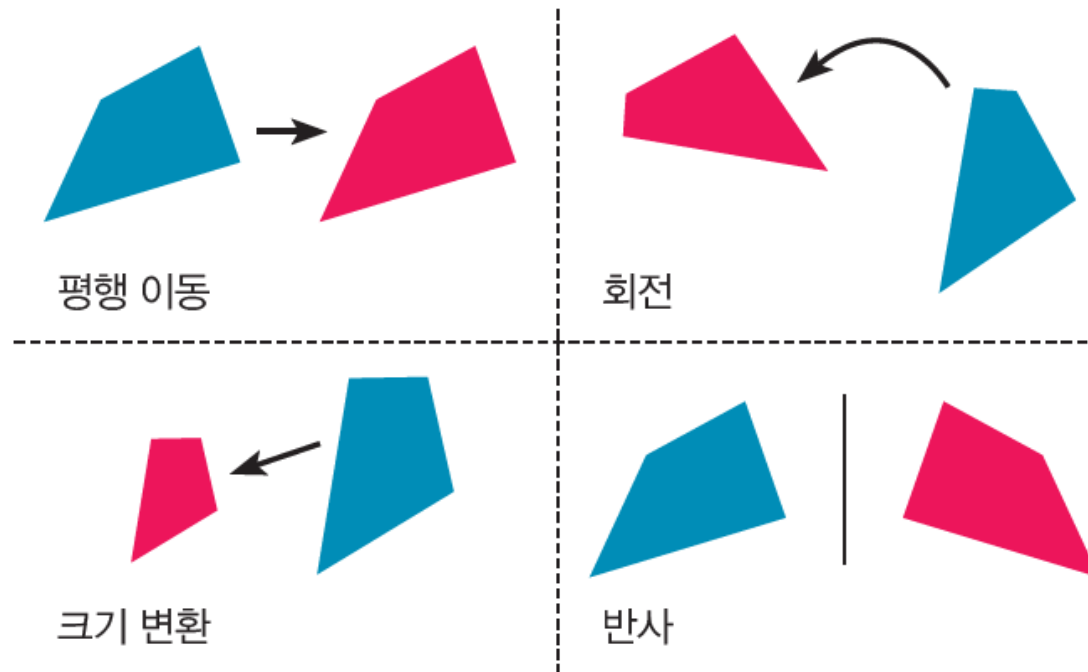
8.1 어파인 변환

8.2 투시 변환

8.1 어파인 변환

영상의 기하학적 변환이란

- 영상의 기하학적 변환(geometric transformation)은 영상을 이동하거나 영상의 모양을 변형하는 영상처리 방법

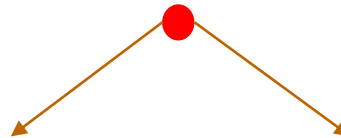


기하학적 변환 vs. 픽셀기반 변환

- 기하학적변환 : 모양이 변경되고 픽셀값은 그대로임
- 픽셀기반 변환 : 모양은 그대로 픽셀값이 변경됨(밝기, 명암비 조절 등)



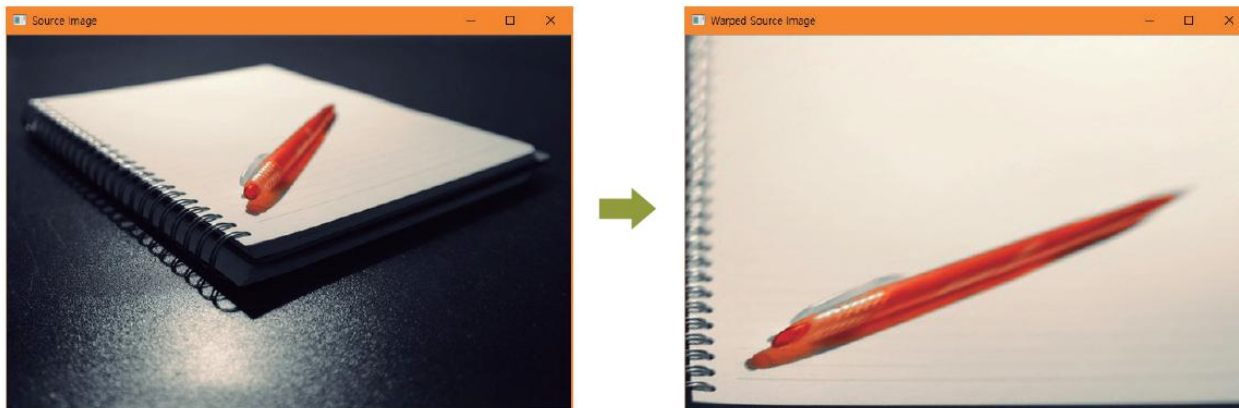
픽셀(pixel)



픽셀값 : (B, G, R) 좌표: (x, y)

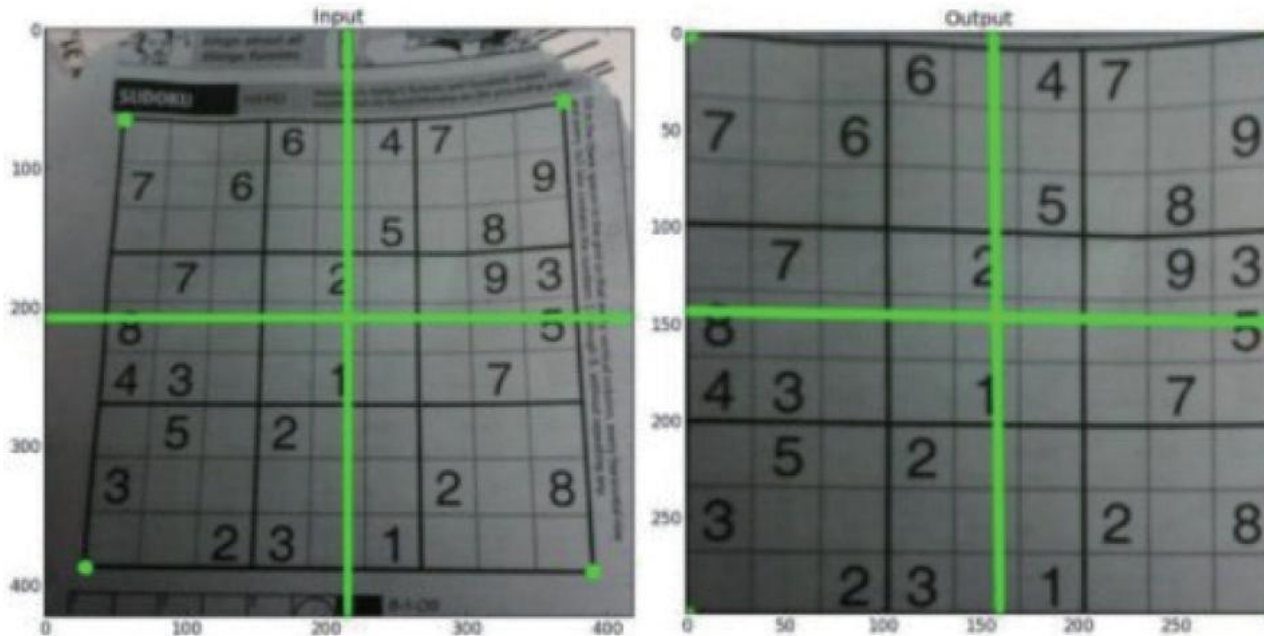
기하학적 변환의 종류

- 어파인(affine) 변환
 - 평행이동(translation) , 크기변환(scaling), 회전(rotation) 등
- 투시 변환
 - 직사각형 형태의 영상을 임의의 볼록 사각형 형태로 변경하는 변환



기하학적 변환의 이유

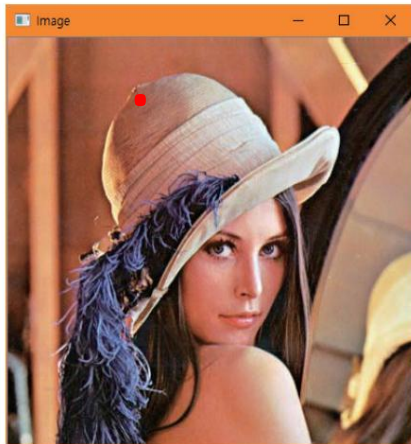
- 관측과 렌즈의 이상에 의해 발생하는 왜곡을 보정
- 서로 다른 센서로부터 얻은 영상들을 합칠 때
- 영화 등에서 특수효과



출처: OpenCV 튜토리얼

기하학적 변환(geometric transform)

- 영상의 밝기 및 명암비 조절, 필터링 등은 픽셀 위치는 고정된 상태에서 **픽셀 값을 변경**하였지만 기하학적 변환은 픽셀 값은 그대로 유지하면서 **위치를 변경하는 작업**임
- 입력 영상에서 (x, y) 좌표의 픽셀을 결과 영상의 (x', y') 좌표로 변환하는 방법은 보통 다음과 같이 고유의 함수 형태로 표현됨



좌표: (x, y)



좌표: (x', y')

$$\begin{cases} x' = f_1(x, y) \\ y' = f_2(x, y) \end{cases}$$

어파인 변환

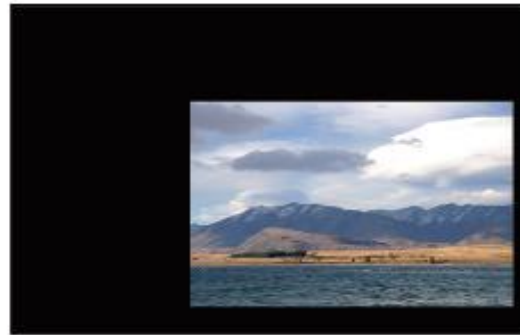
- 앞 수식에서 f_1 과 f_2 는 각각 x 와 y 를 입력으로 받아 결과 영상에서의 픽셀 좌표를 출력하는 함수임
- 함수 f_1 과 f_2 를 어떻게 정의하는지에 따라 영상의 크기를 변경할 수도 있고 영상을 회전시킬 수도 있음
- 기하학적 변환 중에서 **어파인 변환(affine transformation)**은 영상을 평행 이동시키거나 회전, 크기 변환, 전단변환 등을 통해 만들 수 있는 변환을 통칭함
- 어파인 변환의 특징
 - 직선은 그대로 직선으로 변환되고, 직선 간의 길이 비율과 평행 관계가 그대로 유지됨
 - 직사각형 형태의 영상은 어파인 변환에 의해 평행사변형에 해당하는 모양으로 변경됨

어파인 변환

- 각각의 변환 결과 영상 크기는 입력 영상 크기와 같게 설정하였고, 원한다면 결과 영상 크기는 변경할 수 있음



(a)



(b)



(c)



(d)



(e)



(f)

그림 8-1 다양한 어파인 변환의 예

어파인 변환

- 어파인 변환에 의해 입력 영상의 좌표 (x, y) 가 결과 영상의 좌표 (x', y') 로 이동하는 수식은 다음과 같이 1차 다항식으로 표현함

$$\begin{cases} x' = f_1(x, y) = ax + by + c \\ y' = f_2(x, y) = dx + ey + f \end{cases}$$

- 앞 수식에서 a, b, c, d, e, f 가 어파인 변환을 결정하는 여섯 개의 파라미터임
- 앞과 같이 두 개의 수식으로 표현된 어파인 변환은 행렬을 이용하여 하나의 수식으로 표현할 수 있음

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ d & e \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} c \\ f \end{pmatrix}$$

어파인 변환

- 수학적 편의를 위해 입력 영상의 좌표 (x, y) 에 가상의 좌표 1을 하나 추가하여 $(x, y, 1)$ 형태로 바꾸면, 앞 행렬 수식을 다음과 같이 하나의 행렬 곱셈 형태로 바꿀 수 있음

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

- 앞 수식에서 여섯 개의 파라미터로 구성된 2×3 행렬을 어파인 변환 행렬(affine transformation matrix)이라고 부름
- 어파인 변환은 2×3 실수형 행렬 하나로 표현할 수 있음

$$\begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix}$$

어파인 변환

- 입력 영상과 어파인 변환 결과 영상으로부터 어파인 변환 행렬을 구하기 위해서는 최소 3점의 이동 관계를 알아야 함
- 점 하나의 이동 관계로부터 x 좌표와 y 좌표에 대한 변환 수식 2개를 얻고, 점 3개의 이동 관계로부터 총 6개의 방정식을 구함

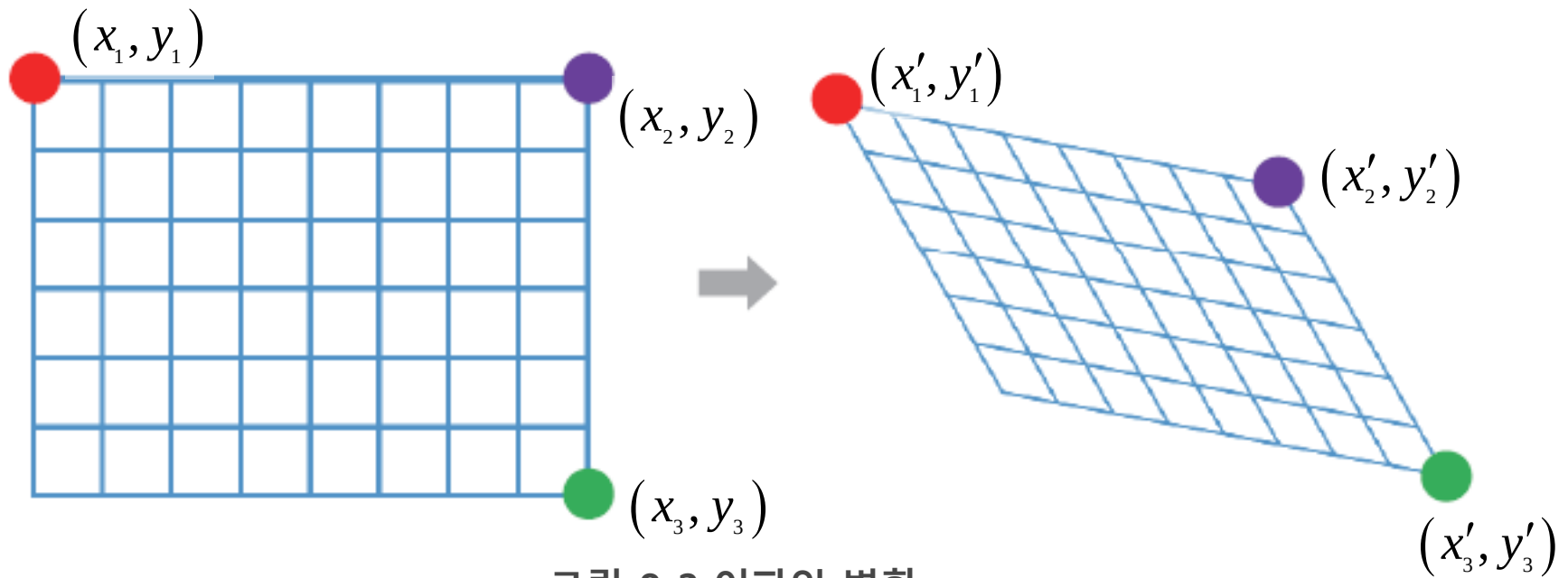


그림 8-2 어파인 변환

어파인 변환

- 어파인 변환에 의해 직사각형 영상은 평행사변형 형태로 변환되므로 입력 영상의 좌측 하단 모서리 점이 이동하는 위치는 자동으로 결정됨
- 어파인 변환은 점 3개의 이동 관계만으로 정의할 수 있음

$$\begin{pmatrix} x'_1 \\ y'_1 \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix} \begin{pmatrix} x_1 \\ y_1 \\ 1 \end{pmatrix} = \begin{pmatrix} ax_1 + by_1 + c \\ dx_1 + ey_1 + f \end{pmatrix}$$

$$\begin{pmatrix} x'_2 \\ y'_2 \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix} \begin{pmatrix} x_2 \\ y_2 \\ 1 \end{pmatrix} = \begin{pmatrix} ax_2 + by_2 + c \\ dx_2 + ey_2 + f \end{pmatrix}$$

$$\begin{pmatrix} x'_3 \\ y'_3 \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix} \begin{pmatrix} x_3 \\ y_3 \\ 1 \end{pmatrix} = \begin{pmatrix} ax_3 + by_3 + c \\ dx_3 + ey_3 + f \end{pmatrix}$$



$$x'_1 = ax_1 + by_1 + c$$

$$y'_1 = dx_1 + ey_1 + f$$

$$x'_2 = ax_2 + by_2 + c$$

$$y'_2 = dx_2 + ey_2 + f$$

$$x'_3 = ax_3 + by_3 + c$$

$$y'_3 = dx_3 + ey_3 + f$$

어파인 변환

```
Mat getAffineTransform(const Point2f src[], const Point2f dst[]);
Mat getAffineTransform(InputArray src, InputArray dst);
```

- **src** 입력 영상에서 세 점의 좌표
- **dst** 결과 영상에서 세 점의 좌표
- **반환값** 2×3 어파인 변환 행렬, CV_64FC1

- getAffineTransform() 함수는 입력 영상에서 3점의 좌표와 이 점들을 변환한 결과 영상의 좌표 3개를 입력으로 받아 2×3 어파인 변환 행렬을 계산함

$$\begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix}$$

어파인 변환

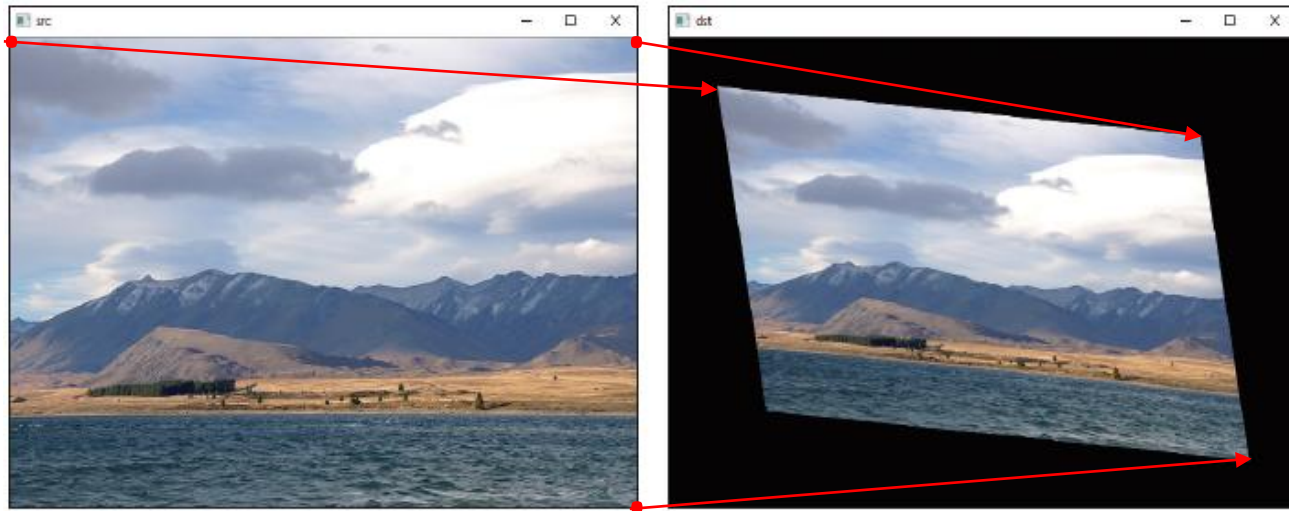
```
void warpAffine(InputArray src, OutputArray dst,
               InputArray M, Size dsize,
               int flags = INTER_LINEAR,
               int borderMode = BORDER_CONSTANT,
               const Scalar& borderValue = Scalar());
```

- **src** 입력 영상
- **dst** 결과 영상. src와 같은 타입이고, 크기는 dsize에 의해 결정됩니다.
- **M** 2×3 어파인 변환 행렬
- **dsize** 결과 영상 크기
- **flags** 보간법 알고리즘. 만약 OR 연산자를 이용하여 WARP_INVERSE_MAP 플래그를 함께 지정하면 역방향으로 변환을 수행합니다.
- **borderMode** 가장자리 픽셀 확장 방식. BorderTypes 열거형 상수 중 하나를 지정합니다. 만약 BORDER_TRANSPARENT를 지정하면 입력 영상의 픽셀 값이 복사되지 않는 영역은 dst 픽셀 값을 그대로 유지합니다.
- **borderValue** borderMode가 BORDER_CONSTANT일 때 사용할 상수 값. 기본값으로 검은색이 지정되어 있습니다.

- 2×3 어파인 변환 행렬을 이용하여 입력 영상을 어파인 변환한 결과 영상을 생성해줌

코드8-1: 어파인 변환

- 3점에 대한 변환 관계로부터 어파인 변환 행렬을 구하고, 이를 이용하여 실제 영상을 어파인 변환하는 예제
- 입력 영상에서 3점은 좌측 상단, 우측 상단, 우측 하단의 모서리 점을 선택하고, 이 점들이 변환된 결과 영상에서의 위치는 임의로 지정함



$(0, 0) \rightarrow (50, 50)$

$(src.cols - 1, 0) \rightarrow (src.cols - 100, 100)$

$(src.cols - 1, src.rows - 1) \rightarrow (src.cols - 50, src.rows - 50)$

코드8-1: 어파인 변환

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main(void)
{
    Mat src = imread("tekapo.bmp");
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1;}

    Point2f srcPts[3], dstPts[3];
    srcPts[0] = Point2f(0, 0);
    srcPts[1] = Point2f(src.cols - 1, 0);
    srcPts[2] = Point2f(src.cols - 1, src.rows - 1);
    dstPts[0] = Point2f(50, 50);
    dstPts[1] = Point2f(src.cols - 100, 100);
    dstPts[2] = Point2f(src.cols - 50, src.rows - 50);
```

코드8-1: 어파인 변환

```
Mat M = getAffineTransform(srcPts, dstPts);  
cout << M << endl;
```

```
Mat dst;  
warpAffine(src, dst, M, Size());
```

```
imshow("src", src);  
imshow("dst", dst);
```

```
waitKey();  
return 0;
```

```
}
```

코드8-1: 어파인 변환

```
D:\Users\2sungryul\Dropbox\Lecture\2021년1학기\컴퓨터비전\강의노트\O...  
[0.7668231611893583, 0.104384133611691, 50;  
0.0782472613458529, 0.6889352818371608, 50]
```

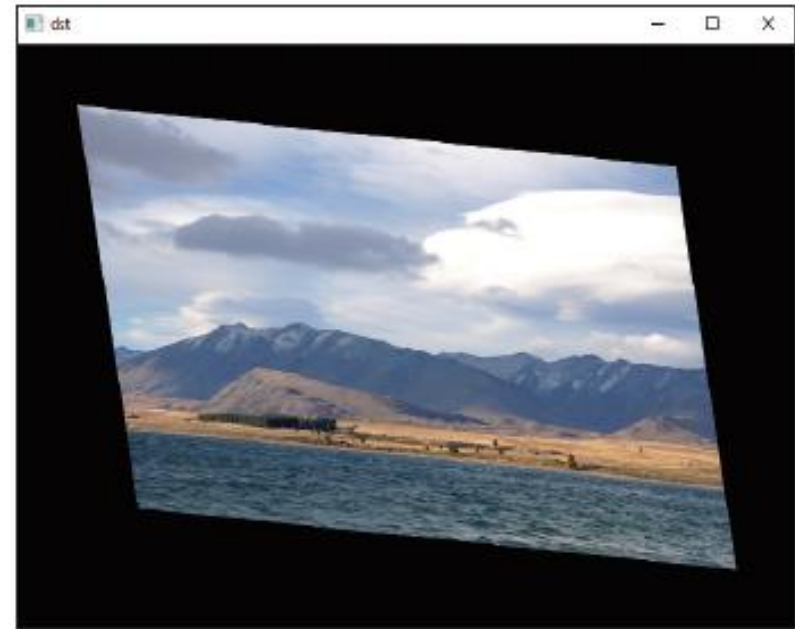
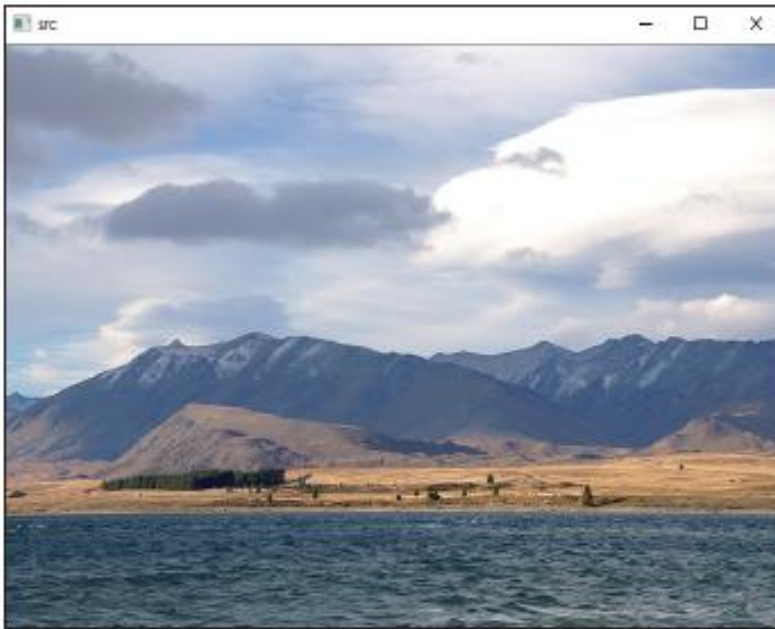


그림 8-3 영상의 어파인 변환 예제 실행 결과

어파인 변환

```
void transform(InputArray src, OutputArray dst, InputArray m);
```

- **src** 입력 행렬 또는 vector<Point2f>. 점의 좌표를 다채널로 표현합니다.
- **dst** 출력 행렬 또는 vector<Point2f>
- **m** 변환 행렬. 2×2 또는 2×3 실수형 행렬

- 참고로 어파인 변환 행렬을 가지고 있을 때, 영상 전체를 어파인 변환하는 것이 아니라 일부 점들이 어느 위치로 이동하는지를 알고 싶다면 transform() 함수를 사용할 수 있음
- 다음 소스 코드를 실행하면 dst 변수에 이동된 점들의 좌표가 저장됨

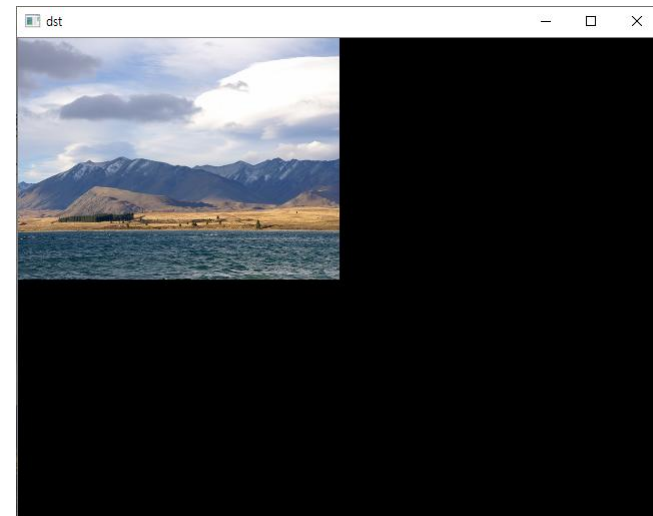
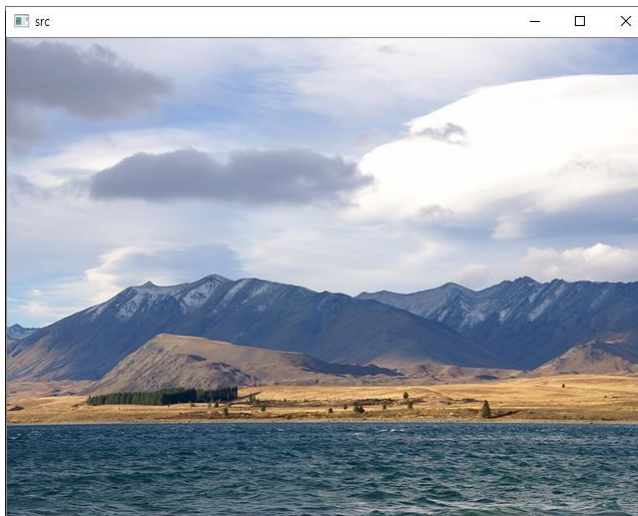
```
vector<Point2f> src = { Point2f(100, 20), Point2f(200, 50) };
vector<Point2f> dst;
```

```
transform(src, dst, M);
```

실습과제 1

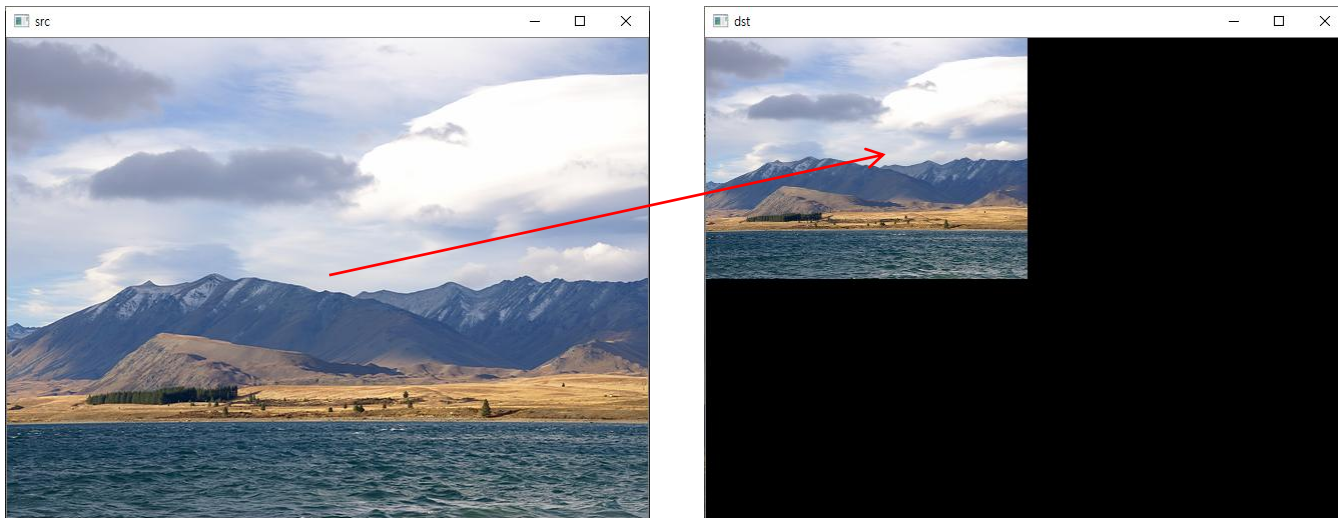
- 아래 그림처럼 입력영상의 크기를 $\frac{1}{2}$ 로 변환하는 어파인 변환 행렬 M을 구하고 입력, 결과영상을 출력하시오.
- 힌트 : 3개 점(좌측상단, 우측상단, 우측하단)의 변환 관계를 구한다.

```
Microsoft Visual Studio 디버그 콘솔
[0.5007824726134585, 0, 0;
0, 0.5010438413361169, 0]
```



실습과제2

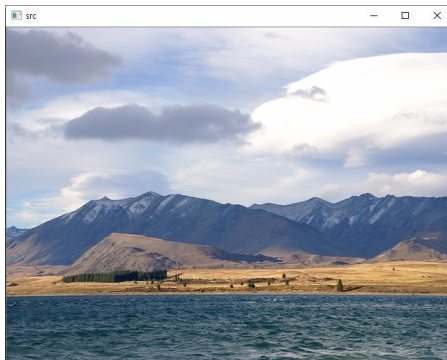
- 실습과제 1번의 변환에서 원본영상의 정중앙점이 결과영상에서 어떤 좌표로 변환되었는지 구하시오. transform함수를 이용할 것



실습과제3

- 아래 그림처럼 입력영상의 크기를 2배로 변환하는 어파인 변환 행렬 M을 구하고 입력, 결과영상을 출력하시오.(dsize에 입력의 2배크기로 설정)
- 힌트 : 3개 점(좌측상단, 우측상단, 우측하단)의 변환 관계를 구한다.

```
D:\Users\W2sungryul\Dropbox\Lecture\2021년...
[2.003129890453834, 0, 0;
0, 2.004175365344468, 0]
```



실습과제4

- 카메라로부터 입력 받은 동영상을 출력하고 키보드 a를 누르면 2배 확대하여 출력하고 b를 누르면 $\frac{1}{2}$ 로 축소하여 출력하고 c를 누르면 정상크기로 출력하는 코드를 작성하시오. q를 누르면 종료하도록 하시오.
- 반복문안에 waitKey함수는 1번만 호출할 것, 2번 이상 호출하면 0점 처리

과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 이클래스 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.